Red Hat OpenShift AI Self-Managed 3.4

Monitoring your AI systems

Monitor your OpenShift AI models for fairness and reliability

Last Updated: 2026-05-15

Red Hat OpenShift AI Self-Managed 3.4 Monitoring your AI systems

Monitor your OpenShift AI models for fairness and reliability

Legal Notice

Abstract

Monitor your machine-learning models in OpenShift AI to ensure they are transparent, fair, and reliable.

Table of Contents

CHAPTER 1. OVERVIEW OF MONITORING YOUR AI SYSTEMS

Use TrustyAI to monitor your models for data drift and bias.

These tools help ensure that your data science and machine learning models are transparent, fair, and reliable.

Configure and set up TrustyAI for your project, and then perform the following checks:

- **Bias:** Check for unfair patterns or biases in data and model predictions to ensure your model's decisions are unbiased.
- **Data drift:** Detect changes in input data distributions over time by comparing the latest real-world data to the original training data. Comparing the data identifies shifts or deviations that could impact model performance, ensuring that the model remains accurate and reliable.

CHAPTER 2. CONFIGURING TRUSTYAI

To configure model monitoring with TrustyAI for data scientists to use in OpenShift AI, a cluster administrator does the following tasks:

- Configure monitoring for the model serving platform
- Enable the TrustyAI component in the Red Hat OpenShift AI Operator
- Configure TrustyAI to use a database, if you want to use your database instead of a PVC for storage with TrustyAI
- Install the TrustyAI service on each project that contains models that the data scientists want to monitor
- (Optional) Configure TrustyAI and KServe RawDeployment mode integration

2.1. CONFIGURING MONITORING FOR YOUR MODEL SERVING PLATFORM

For deploying large models such as large language models (LLMs), use the model serving platform.

+ To configure monitoring for this platform, see Configuring monitoring for the model serving platform.

2.2. ENABLING THE TRUSTYAI COMPONENT

To allow your data scientists to use model monitoring with TrustyAI, you must enable the TrustyAI component in OpenShift AI.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have access to the data science cluster.
- You have installed Red Hat OpenShift AI.

Procedure

1. In the OpenShift console, click **Ecosystem → Installed Operators**.
2. Search for the **Red Hat OpenShift AI** Operator, and then click the Operator name to open the Operator details page.
3. Click the **Data Science Cluster** tab.
4. Click the default instance name (for example, **default-dsc**) to open the instance details page.
5. Click the **YAML** tab to show the instance specifications.
6. In the **spec:components** section, set the **managementState** field for the **trustyai** component to **Managed**:

```
trustyai:
  managementState: Managed
```

7. Click **Save**.

Verification

Check the status of the **trustyai-service-operator** pod:

1. In the OpenShift console, from the **Project** list, select **redhat-ods-applications**.
2. Click **Workloads → Deployments**.
3. Search for the **trustyai-service-operator-controller-manager** deployment. Check the status:
   a. Click the deployment name to open the deployment details page.
   b. Click the **Pods** tab.
   c. View the pod status.
   When the status of the **trustyai-service-operator-controller-manager-*<pod-id>*** pod is **Running**, the pod is ready to use.

2.3. CONFIGURING TRUSTYAI WITH A DATABASE

If you have a relational database in your OpenShift cluster such as MySQL or MariaDB, you can configure TrustyAI to use your database instead of a persistent volume claim (PVC). Using a database instead of a PVC for storage can improve scalability, performance, and data management in TrustyAI. Provide TrustyAI with a database configuration secret before deployment. You can create a secret or specify the name of an existing Kubernetes secret within your project.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have enabled the TrustyAI component, as described in Enabling the TrustyAI component.
- The data scientist has created a project, as described in Creating a project, that contains the models that the data scientist wants to monitor.
- If you are configuring the TrustyAI service with an external MySQL database, your database must already be in your cluster and use at least MySQL version 5.x. However, Red Hat recommends that you use MySQL version 8.x.
- If you are configuring the TrustyAI service with a MariaDB database, your database must already be in your cluster and use MariaDB version 10.3 or later. However, Red Hat recommends that you use at least MariaDB version 10.5.

NOTE

The transport security layer (TLS) protocol does not work with the MariaDB operator 0.29 or later versions.

The MariaDB operator for **s390x** is not supported at this time.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Optional: If you want to use a TLS connection between TrustyAI and the database, create a TrustyAI service database TLS secret that uses the same certificates that you want to use for the database.

   a. Create a YAML file to contain your TLS secret and add the following code:

```
apiVersion: v1
kind: Secret
metadata:
  name: <service_name>-db-tls
type: kubernetes.io/tls
data:
  tls.crt: |
    <TLS CERTIFICATE>

  tls.key: |
    <TLS KEY>
```

   b. Save the file with the file name ***<service_name>*-db-tls.yaml**. For example, if your service name is **trustyai-service**, save the file as **trustyai-service-db-tls.yaml**.

   c. Apply the YAML file in the project that contains the models that the data scientist wants to monitor:

```
$ oc apply -f <service_name>-db-tls.yaml -n <project_name>
```

3. Create a secret (or specify an existing one) that has your database credentials.

   a. Create a YAML file to contain your secret and add the following code:

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:
  databaseKind: <mariadb> 1
  databaseUsername: <TrustyAI_username> 2
  databasePassword: <TrustyAI_password> 3
  databaseService: mariadb-service 4
```

```
databasePort: 3306 5
databaseGeneration: update 6
databaseName: trustyai_service 7
```

1. The only currently supported **databaseKind** value is **mariadb**.
2. The username you want TrustyAI to use when interfacing with the database.
3. The password that TrustyAI must use when connecting to the database.
4. The Kubernetes (K8s) service that TrustyAI must use when connecting to the database (the default **mariadb**) .
5. The port that TrustyAI must use when connecting to the database (default is 3306).
6. The database schema generation strategy to be used by TrustyAI. It is the setting for the **quarkus.hibernate-orm.database.generation** argument, which determines how TrustyAI interacts with the database on its initial connection. Set to **none**, **create**, **drop-and-create**, **drop**, **update**, or **validate**.
7. The name of the individual database within the database service that the username and password authenticate to, as well as the specific database name that TrustyAI should read and write to on the database server.

b. Save the file with the file name **db-credentials.yaml**. You will need this name later when you install or change the TrustyAI service.

c. Apply the YAML file in the project that contains the models that the data scientist wants to monitor:

```
$ oc apply -f db-credentials.yaml -n <project_name>
```

4. If you are installing TrustyAI for the first time on a project, continue to Installing the TrustyAI service for a project.
   If you already installed TrustyAI on a project, you can migrate the existing TrustyAI service from using a PVC to using a database.

a. Create a YAML file to update the TrustyAI service custom resource (CR) and add the following code:

```
apiVersion: trustyai.opendatahub.io/v1
kind: TrustyAIService
metadata:
  annotations:
    trustyai.opendatahub.io/db-migration: "true" 1
  name: trustyai-service 2
spec:
  storage:
    format: "DATABASE" 3
    folder: "/inputs" 4
      size: "1Gi" 5
    databaseConfigurations: <database_secret_credentials> 6
  data:
```

```
    filename: "data.csv" 7
  metrics:
    schedule: "5s" 8
```

1. Set to **true** to prompt the migration from PVC to database storage.
2. The name of the TrustyAI service instance.
3. The storage format for the data. Set this field to **DATABASE**.
4. The location within the PVC where you were storing the data. This must match the value specified in the existing CR.
5. The size of the data to request.
6. The name of the secret with your database credentials that you created in an earlier step. For example, **db-credentials**.
7. The suffix for the existing stored data files. This must match the value specified in the existing CR.
8. The interval at which to calculate the metrics. The default is **5s**. The duration is specified with the ISO-8601 format. For example, **5s** for 5 seconds, **5m** for 5 minutes, and **5h** for 5 hours.

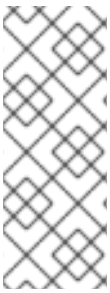
b. Save the file. For example, **trustyai_crd.yaml**.

c. Apply the new TrustyAI service CR to the project that contains the models that the data scientist wants to monitor:

```
$ oc apply -f trustyai_crd.yaml -n <project_name>
```

2.4. INSTALLING THE TRUSTYAI SERVICE FOR A PROJECT

Install the TrustyAI service on a project to provide access to its features for all models deployed within that project. An instance of the TrustyAI service is required for each project, or namespace, that contains models that the data scientists want to monitor.



NOTE

Install only one instance of the TrustyAI service in a project. Multiple instances in the same project can result in unexpected behavior.

TrustyAI only supports models deployed with OpenVINO Model Server (OVMS). Non-OVMS models are not supported. Installing TrustyAI into a namespace where non-OVMS models are deployed can cause errors in the TrustyAI service.

2.4.1. Installing the TrustyAI service by using the CLI

You can use the OpenShift CLI (**oc**) to install an instance of the TrustyAI service.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have configured monitoring for the model serving platform, as described in Configuring monitoring for your model serving platform.
- You have enabled the TrustyAI component, as described in Enabling the TrustyAI component.
- If you are using TrustyAI with a database instead of PVC, you have configured TrustyAI to use the database, as described in Configuring TrustyAI with a database.
- The data scientist has created a project, as described in Creating a project, that contains the models that the data scientist wants to monitor.

Procedure

1. Open a new terminal window.
2. Follow these steps to log in to your OpenShift cluster as a cluster administrator:
   a. In the OpenShift web console, click your user name and select **Copy login command**.
   b. After you have logged in, click **Display token**.
   c. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

   ```
   $ oc login --token=<token> --server=<openshift_cluster_url>
   ```
3. Navigate to the project that contains the models that the data scientist wants to monitor.

   ```
   oc project <project_name>
   ```

   For example:

   ```
   oc project my-project
   ```
4. Create a **TrustyAIService** custom resource (CR) file, for example **trustyai_crd.yaml**:

   Example CR file for TrustyAI using a database

   ```
   apiVersion: trustyai.opendatahub.io/v1
   kind: TrustyAIService
   metadata:
     name: trustyai-service 1
   spec:
     storage:
       format: "DATABASE" 2
       size: "1Gi" 3
   ```

```
  databaseConfigurations: <database_secret_credentials> 4
 metrics:
  schedule: "5s" 5
```

1. The name of the TrustyAI service instance.
2. The storage format for the data, either **DATABASE** or **PVC** (persistent volume claim). Red Hat recommends that you use a database setup for better scalability, performance, and data management in TrustyAI.
3. The size of the data to request.
4. The name of the secret with your database credentials that you created in Configuring TrustyAI with a database. For example, **db-credentials**.
5. The interval at which to calculate the metrics. The default is **5s**. The duration is specified with the ISO-8601 format. For example, **5s** for 5 seconds, **5m** for 5 minutes, and **5h** for 5 hours.

Example CR file for TrustyAI using a PVC

```
apiVersion: trustyai.opendatahub.io/v1
kind: TrustyAIService
metadata:
  name: trustyai-service 1
spec:
  storage:
   format: "PVC" 2
   folder: "/inputs" 3
   size: "1Gi" 4
  data:
   filename: "data.csv" 5
   format: "CSV" 6
  metrics:
   schedule: "5s" 7
   batchSize: 5000 8
```

1. The name of the TrustyAI service instance.
2. The storage format for the data, either **DATABASE** or **PVC** (persistent volume claim).
3. The location within the PVC where you want to store the data.
4. The size of the PVC to request.
5. The suffix for the stored data files.
6. The format of the data. Currently, only comma-separated value (CSV) format is supported.
7. The interval at which to calculate the metrics. The default is **5s**. The duration is specified with the ISO-8601 format. For example, **5s** for 5 seconds, **5m** for 5 minutes, and **5h** for 5 hours.

8 (Optional) The observation's historical window size to use for metrics calculation. The default is **5000**, which means that the metrics are calculated using the 5,000 latest

5. Add the TrustyAI service's CR to your project:

```
oc apply -f trustyai_crd.yaml
```

This command returns output similar to the following:

```
trusty-service created
```

Verification

Verify that you installed the TrustyAI service:

```
oc get pods | grep trustyai
```

You should see a response similar to the following:

```
trustyai-service-5d45b5884f-96h5z          1/1     Running
```

2.5. ENABLING TRUSTYAI INTEGRATION WITH KSERVE RAWDEPLOYMENT

To use the TrustyAI service with KServe RawDeployment mode, you must first update the KServe ConfigMap, then create another ConfigMap in your model's namespace to hold the Certificate Authority (CA) certificate.

Prerequisites

- You have installed Red Hat OpenShift AI.
- You have cluster administrator privileges for your OpenShift AI cluster.
- You have access to a data science cluster that has TrustyAI enabled.
- You have enabled the model serving platform.

Procedure

1. Update the KServe ConfigMap (**inferenceservice-config**) in the OpenShift AI UI:
   a. From the OpenShift console, click **Workloads → ConfigMaps**.
   b. From the project drop-down list, select the **redhat-ods-applications** namespace.
   c. Find the **inferenceservice-config** ConfigMap.
   d. Click the options menu (⋮) for that ConfigMap, and then click **Edit ConfigMap**.
   e. Add the following parameters to the logger key:

```
"caBundle": "kserve-logger-ca-bundle",
"caCertFile": "service-ca.crt",
"tlsSkipVerify": false
```

f. Click **Save**.

2. Create a ConfigMap in your model's namespace to hold the CA certificate:

a. Click **Create Config Map**.

b. Enter the following code in the created ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: kserve-logger-ca-bundle
  namespace: <your-model-namespace>
  annotations:
    service.beta.openshift.io/inject-cabundle: "true"
data: {}
```

3. Click **Save**.

NOTE

The **caBundle** name can be any valid Kubernetes name, as long as it matches the name you used in the KServe ConfigMap. The **caCertFile** needs to match the cert name available in the CA bundle.

Verification

When you send inferences to your KServe Raw model, TrustyAI acknowledges the data capture in the output logs.

NOTE

If you do not observe any data on the Trusty AI logs, complete these configuration steps and redeploy the pod.

CHAPTER 3. SETTING UP TRUSTYAI FOR YOUR PROJECT

To set up model monitoring with TrustyAI for a project, a data scientist does the following tasks:

- Authenticate the TrustyAI service
- Send training data to TrustyAI for bias or data drift monitoring
- Label your data fields (optional)

After setting up, a data scientist can create and view bias and data drift metrics for deployed models.

3.1. AUTHENTICATING THE TRUSTYAI SERVICE

To access TrustyAI service external endpoints, you must provide OAuth proxy (oauth-proxy) authentication. You must obtain a user token, or a token from a service account with sufficient privileges, and then pass the token to the TrustyAI service when using **curl** commands.

Prerequisites

- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- Your OpenShift cluster administrator added you as a user to the OpenShift cluster and has installed the TrustyAI service for the project that contains the deployed models.

Procedure

1. Open a new terminal window.
2. Follow these steps to log in to your OpenShift cluster:
   a. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
   b. After you have logged in, click **Display token**.
   c. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

3. Enter the following command to set a user token variable on OpenShift:

```
export TOKEN=$(oc whoami -t)
```

Verification

- Enter the following command to check the user token variable:

```
echo $TOKEN
```

Next step

When running **curl** commands, pass the token to the TrustyAI service using the Authorization header. For example:

```
curl -H "Authorization: Bearer $TOKEN" $TRUSTY_ROUTE
```

3.2. UPLOADING TRAINING DATA TO TRUSTYAI

Upload training data to use with TrustyAI for bias monitoring or data drift detection.

Prerequisites

- Your cluster administrator added you as a user to the OpenShift cluster and has installed the TrustyAI service for the project that contains the deployed models.
- You have model training data to upload.
- You authenticated the TrustyAI service as described in Authenticating the TrustyAI service.

Procedure

1. Set the **TRUSTY_ROUTE** variable to the external route for the TrustyAI service in your project:

```
TRUSTY_ROUTE=https://$(oc get route/trustyai-service --template={{.spec.host}})
```

2. Send the training data to the **/data/upload** endpoint:

```
curl -sk $TRUSTY_ROUTE/data/upload  \
  --header 'Authorization: Bearer ${TOKEN}' \
  --header 'Content-Type: application/json' \
  -d @data/training_data.json
```

The following message is displayed if the upload was successful: **1000 datapoints successfully added to gaussian-credit-model data**.

Verification

- Verify that TrustyAI has received the data via the **/info** endpoint by inputting this query:

```
curl -H 'Authorization: Bearer ${TOKEN}' \
    $TRUSTY_ROUTE/info | jq ".[0].data"
```

The output returns a json file containing the following information for the model:

  - The names, data types, and positions of fields in the input and output.
  - The observed values that these fields take. This value is usually **null** because there are too many unique feature values to enumerate.
  - The total number of input-output pairs observed. It should be **1000**.

3.3. SENDING TRAINING DATA TO TRUSTYAI

To use TrustyAI for bias monitoring or data drift detection, you must send training data for your model to TrustyAI.

Prerequisites

- Your OpenShift cluster administrator added you as a user to the OpenShift cluster and has installed the TrustyAI service for the project that contains the deployed models.
- You authenticated the TrustyAI service as described in Authenticating the TrustyAI service.
- You have uploaded model training data to TrustyAI.
- Your deployed model is registered with TrustyAI.

Procedure

1. Verify that the TrustyAI service has registered your deployed model:
   a. In the OpenShift console, go to **Workloads → Pods**.
   b. From the project list, select the project that contains your deployed model.
   c. Inspect the **InferenceService** for your deployed model. For example, run the following command:

```
oc describe inferenceservice my-model -n my-namespace
```

   d. When inspecting the **InferenceService**, you should see the following field in the specification:

```
Logger:
    # ...
    Mode:  all
    URL:   https://trustyai-service.my-namespace.svc.cluster.local
```

2. Set the **TRUSTY_ROUTE** variable to the external route for the TrustyAI service pod:

```
TRUSTY_ROUTE=https://$(oc get route/trustyai-service --template={{.spec.host}})
```

3. Get the inference endpoints for the deployed model, as described in Accessing the inference endpoint for a deployed model.
4. Send data to this endpoint. For more information, see the KServe v2 Inference Protocol documentation.

Verification

Follow these steps to view cluster metrics and verify that TrustyAI is receiving data.

1. Log in to the OpenShift web console.
2. Switch to the **Developer** perspective.
3. In the left menu, click **Observe**.
4. On the **Metrics** page, click the **Select query** list and then select **Custom query**.

5. In the **Expression** field, enter **trustyai_model_observations_total** and press Enter. Your model should be listed and reporting observed inferences.

6. Optional: Select a time range from the list above the graph. For example, select **5m**.

3.4. LABELING DATA FIELDS

After you send model training data to TrustyAI, you might want to apply a set of name mappings to your inputs and outputs so that the field names are meaningful and easier to work with.

Prerequisites

- Your OpenShift cluster administrator added you as a user to the OpenShift cluster and has installed the TrustyAI service for the project that contains the deployed models.
- You sent training data to TrustyAI as described in Sending training data to TrustyAI.

Procedure

1. Open a new terminal window.
2. Follow these steps to log in to your OpenShift cluster:
   a. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
   b. After you have logged in, click **Display token**.
   c. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

   ```
   $ oc login --token=<token> --server=<openshift_cluster_url>
   ```

3. In the OpenShift CLI (**oc**), get the route to the TrustyAI service:

   ```
   TRUSTY_ROUTE=https://$(oc get route/trustyai-service --template={{.spec.host}})
   ```

4. To examine TrustyAI's model metadata, query the /**info** endpoint:

   ```
   curl -H "Authorization: Bearer $TOKEN" $TRUSTY_ROUTE/info | jq ".[0].data"
   ```

   This outputs a JSON file containing the following information for each model:

   - The names, data types, and positions of input fields and output fields.
   - The observed field values.
   - The total number of input-output pairs observed.

5. Use **POST /info/names** to apply name mappings to the fields, similar to the following example. Change the **model-name**, **original-name**, and **Prediction** values to those used in your model. Change the **New name** values to the labels that you want to use.

   ```
   curl -sk -H "Authorization: Bearer $TOKEN" -X POST --location
   $TRUSTY_ROUTE/info/names \
   ```

```
-H "Content-Type: application/json"   \
-d "{
  \"modelId\": \"model-name\",
  \"inputMapping\":
    {
      \"original-name-0\": \"New name 0\",
      \"original-name-1\": \"New name 1\",
      \"original-name-2\": \"New name 2\",
      \"original-name-3\": \"New name 3\",
    },
  \"outputMapping\": {
    \"predict-0\": \"Prediction 0\"
  }
}"
```

For another example, see https://github.com/trustyai-explainability/odh-trustyai-demos/blob/main/2-BiasMonitoring/kserve-demo/scripts/apply_name_mapping.sh.

Verification

A "Feature and output name mapping successfully applied" message is displayed.

CHAPTER 4. MONITORING MODEL BIAS

As a data scientist, you might want to monitor your machine learning models for bias. This means monitoring for algorithmic deficiencies that might skew the outcomes or decisions that the model produces. Importantly, this type of monitoring helps you to ensure that the model is not biased against particular protected groups or features.

Red Hat OpenShift AI provides a set of metrics that help you to monitor your models for bias. You can use the OpenShift AI interface to choose an available metric and then configure model-specific details such as a protected attribute, the privileged and unprivileged groups, the outcome you want to monitor, and a threshold for bias. You then see a chart of the calculated values for a specified number of model inferences.

For more information about the specific bias metrics, see Using bias metrics.

4.1. CREATING A BIAS METRIC

To monitor a deployed model for bias, you must first create bias metrics. When you create a bias metric, you specify details relevant to your model such as a protected attribute, privileged and unprivileged groups, a model outcome and a value that you want to monitor, and the acceptable threshold for bias.

For information about the specific bias metrics, see Using bias metrics.

For the complete list of TrustyAI metrics, see TrustyAI service API.

You can create a bias metric for a model by using the OpenShift AI dashboard or by using the OpenShift CLI (**oc**).

4.1.1. Creating a bias metric by using the dashboard

You can use the OpenShift AI dashboard to create a bias metric for a model.

Prerequisites

- You are familiar with the bias metrics that you can use with OpenShift AI and how to interpret them.
- You are familiar with the specific data set schema and understand the names and meanings of the inputs and outputs.
- Your OpenShift cluster administrator added you as a user to the OpenShift cluster and has installed the TrustyAI service for the project that contains the deployed models.
- You set up TrustyAI for your project, as described in Setting up TrustyAI for your project.

Procedure

1. Optional: To set the **TRUSTY_ROUTE** variable, follow these steps.
   a. In a terminal window, log in to the OpenShift cluster where OpenShift AI is deployed.

      ```
      oc login
      ```

   b. Set the **TRUSTY_ROUTE** variable to the external route for the TrustyAI service pod.

```
TRUSTY_ROUTE=https://$(oc get route/trustyai-service --template={{.spec.host}})
```

2. In the left menu of the OpenShift AI dashboard, click **AI hub → Deployments**.
3. On the **Deployments** page, select your project from the drop-down list.
4. Click the name of the model that you want to configure bias metrics for.
5. On the metrics page for the model, click the **Model bias** tab.
6. Click **Configure**.
7. In the **Configure bias metrics** dialog, complete the following steps to configure bias metrics:
   a. In the **Metric name** field, type a unique name for your bias metric. Note that you cannot change the name of this metric later.
   b. From the **Metric type** list, select one of the metrics types that are available in OpenShift AI.
   c. In the **Protected attribute** field, type the name of an attribute in your model that you want to monitor for bias.

      TIP

      You can use a **curl** command to query the metadata endpoint and view input attribute names and values. For example: **curl -H "Authorization: Bearer $TOKEN" $TRUSTY_ROUTE/info | jq ".[0].data.inputSchema"**

   d. In the **Privileged value** field, type the name of a privileged group for the protected attribute that you specified.
   e. In the **Unprivileged value** field, type the name of an unprivileged group for the protected attribute that you specified.
   f. In the **Output** field, type the name of the model outcome that you want to monitor for bias.

      TIP

      You can use a **curl** command to query the metadata endpoint and view output attribute names and values. For example: **curl -H "Authorization: Bearer $TOKEN" $TRUSTY_ROUTE/info | jq ".[0].data.outputSchema"**

   g. In the **Output value** field, type the value of the outcome that you want to monitor for bias.
   h. In the **Violation threshold** field, type the bias threshold for your selected metric type. This threshold value defines how far the specified metric can be from the fairness value for your metric, before the model is considered biased.
   i. In the **Metric batch size** field, type the number of model inferences that OpenShift AI includes each time it calculates the metric.
8. Ensure that the values you entered are correct.

NOTE

You cannot edit a model bias metric configuration after you create it. Instead, you can duplicate a metric and then edit (configure) it; however, the history of the original metric is not applied to the copy.

9. Click **Configure**.

Verification

- The **Bias metric configuration** page shows the bias metrics that you configured for your model.

Next step

To view metrics, on the **Bias metric configuration** page, click **View metrics** in the upper-right corner.

4.1.2. Creating a bias metric by using the CLI

You can use the OpenShift CLI (**oc**) to create a bias metric for a model.

Prerequisites

- You are familiar with the bias metrics that you can use with OpenShift AI and how to interpret them.
- You are familiar with the specific data set schema and understand the names and meanings of the inputs and outputs.
- Your OpenShift cluster administrator added you as a user to the OpenShift cluster and has installed the TrustyAI service for the project that contains the deployed models.
- You set up TrustyAI for your project, as described in Setting up TrustyAI for your project.

Procedure

1. In a terminal window, log in to the OpenShift cluster where OpenShift AI is deployed.

```
oc login
```

2. Set the **TRUSTY_ROUTE** variable to the external route for the TrustyAI service pod.

```
TRUSTY_ROUTE=https://$(oc get route/trustyai-service --template={{.spec.host}})
```

3. Optionally, get the full list of TrustyAI service endpoints and payloads.

```
curl -H "Authorization: Bearer $TOKEN" --location $TRUSTY_ROUTE/q/openapi
```

4. Use **POST /metrics/group/fairness/spd/request** to schedule a recurring bias monitoring metric with the following syntax and payload structure:
Syntax:

```
curl -sk -H "Authorization: Bearer $TOKEN" -X POST --location
$TRUSTY_ROUTE/metrics/group/fairness/spd/request  \
 --header 'Content-Type: application/json' \
```

```
--data <payload>
```

Payload structure:

modelId

The name of the model to query.

protectedAttribute

The name of the feature that distinguishes the groups that you are checking for fairness.

privilegedAttribute

The suspected favored (positively biased) class.

unprivilegedAttribute

The suspected unfavored (negatively biased) class.

outcomeName

The name of the output that provides the output you are examining for fairness.

favorableOutcome

The value of the **outcomeName** output that describes the favorable or desired model prediction.

batchSize

The number of previous inferences to include in the calculation.

For example:

```
curl -sk -H "Authorization: Bearer $TOKEN" -X POST --location $TRUSTY_ROUTE
/metrics/group/fairness/spd/request \
    --header 'Content-Type: application/json' \
    --data "{
                \"modelId\": \"demo-loan-nn-onnx-alpha\",
                \"protectedAttribute\": \"Is Male-Identifying?\",
                \"privilegedAttribute\": 1.0,
                \"unprivilegedAttribute\": 0.0,
                \"outcomeName\": \"Will Default?\",
                \"favorableOutcome\": 0,
                \"batchSize\": 5000
            }"
```

Verification

The bias metrics request should return output similar to the following:

```
{
  "timestamp":"2023-10-24T12:06:04.586+00:00",
  "type":"metric",
  "value":-0.0029676404469311524,
  "namedValues":null,
  "specificDefinition":"The SPD of -0.002968 indicates that the likelihood of Group:Is Male-
Identifying?=1.0 receiving Outcome:Will Default?=0 was -0.296764 percentage points lower than that
of Group:Is Male-Identifying?=0.0.",
  "name":"SPD",
  "id":"d2707d5b-cae9-41aa-bcd3-d950176cbbaf",
  "thresholds":{"lowerBound":-0.1,"upperBound":0.1,"outsideBounds":false}
}
```

The **specificDefinition** field helps you understand the real-world interpretation of these metric values. For this example, the model is fair over the **Is Male-Identifying?** field, with the rate of positive outcome only differing by about -0.3%.

4.1.3. Duplicating a bias metric

If you want to edit an existing metric, you can duplicate (copy) it in the OpenShift AI interface and then edit the values in the copy. However, note that the history of the original metric is not applied to the copy.

Prerequisites

- You are familiar with the bias metrics that you can use with OpenShift AI and how to interpret them.
- You are familiar with the specific data set schema and understand the names and meanings of the inputs and outputs.
- There is an existing bias metric that you want to duplicate.

Procedure

1. In the left menu of the OpenShift AI dashboard, click **AI hub → Deployments**.
2. On the **Deployments** page, click the name of the model with the bias metric that you want to duplicate.
3. On the metrics page for the model, click the **Model bias** tab.
4. Click **Configure**.
5. On the **Bias metric configuration** page, click the action menu (⋮) next to the metric that you want to copy and then click **Duplicate**.
6. In the **Configure bias metric** dialog, follow these steps:
   a. In the **Metric name** field, type a unique name for your bias metric. Note that you cannot change the name of this metric later.
   b. Change the values of the fields as needed. For a description of these fields, see Creating a bias metric by using the dashboard.
7. Ensure that the values you entered are correct, and then click **Configure**.

Verification

- The **Bias metric configuration** page shows the bias metrics that you configured for your model.

Next step

To view metrics, on the **Bias metric configuration** page, click **View metrics** in the upper-right corner.

4.2. DELETING A BIAS METRIC

You can delete a bias metric for a model by using the OpenShift AI dashboard or by using the OpenShift CLI (**oc**).

4.2.1. Deleting a bias metric by using the dashboard

You can use the OpenShift AI dashboard to delete a bias metric for a model.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- There is an existing bias metric that you want to delete.

Procedure

1. In the left menu of the OpenShift AI dashboard, click **AI hub → Deployments**.
2. On the **Deployments** page, click the name of the model with the bias metric that you want to delete.
3. On the metrics page for the model, click the **Model bias** tab.
4. Click **Configure**.
5. Click the action menu (⋮) next to the metric that you want to delete and then click **Delete**.
6. In the **Delete bias metric** dialog, type the metric name to confirm the deletion.

NOTE

You cannot undo deleting a bias metric.

7. Click **Delete bias metric**

Verification

- The **Bias metric configuration** page does not show the bias metric that you deleted.

4.2.2. Deleting a bias metric by using the CLI

You can use the OpenShift CLI (**oc**) to delete a bias metric for a model.

Prerequisites

- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have a user token for authentication as described in Authenticating the TrustyAI service.
- There is an existing bias metric that you want to delete.

Procedure

1. Open a new terminal window.
2. Follow these steps to log in to your OpenShift cluster:
   a. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
   b. After you have logged in, click **Display token**.
   c. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

   ```
   $ oc login --token=<token> --server=<openshift_cluster_url>
   ```

3. In the OpenShift CLI (**oc**), get the route to the TrustyAI service:

   ```
   TRUSTY_ROUTE=https://$(oc get route/trustyai-service --template={{.spec.host}})
   ```

4. Optional: To list all currently active requests for a metric, use **GET /metrics/{{metric}}/requests**. For example, to list all currently scheduled SPD metrics, type:

   ```
   curl -H "Authorization: Bearer $TOKEN" -X GET --location
   "$TRUSTY_ROUTE/metrics/spd/requests"
   ```

   Alternatively, to list all currently scheduled metric requests, use **GET /metrics/all/requests**.

   ```
   curl -H "Authorization: Bearer $TOKEN" -X GET --location
   "$TRUSTY_ROUTE/metrics/all/requests"
   ```

5. To delete a metric, send an HTTP **DELETE** request to the **/metrics/$METRIC/request** endpoint to stop the periodic calculation, including the id of periodic task that you want to cancel in the payload. For example:

   ```
   curl -H "Authorization: Bearer $TOKEN" -X DELETE --location
   "$TRUSTY_ROUTE/metrics/spd/request" \
     -H "Content-Type: application/json" \
     -d "{
           \"requestId\": \"3281c891-e2a5-4eb3-b05d-7f3831acbb56\"
         }"
   ```

Verification

Use **GET /metrics/{{metric}}/requests** to list all currently active requests for the metric and verify the metric that you deleted is not shown. For example:

```
curl -H "Authorization: Bearer $TOKEN" -X GET --location "$TRUSTY_ROUTE/metrics/spd/requests"
```

4.3. VIEWING BIAS METRICS FOR A MODEL

After you create bias monitoring metrics, you can use the OpenShift AI dashboard to view and update the metrics that you configured.

Prerequisite

Prerequisite

- You configured bias metrics for your model as described in Creating a bias metric.

Procedure

1. In the OpenShift AI dashboard, click **AI hub → Deployments**.
2. On the **Deployments** page, click the name of a model that you want to view bias metrics for.
3. On the metrics page for the model, click the **Model bias** tab.
4. To update the metrics shown on the page, follow these steps:
   a. In the **Metrics to display** section, use the **Select a metric** list to select a metric to show on the page.

NOTE

Each time you select a metric to show on the page, an additional **Select a metric** list is displayed. This enables you to show multiple metrics on the page.

   b. From the **Time range** list in the upper-right corner, select a value.
   c. From the **Refresh interval** list in the upper-right corner, select a value.
   The metrics page shows the metrics that you selected.
5. Optional: To remove one or more metrics from the page, in the **Metrics to display** section, perform one of the following actions:
   - To remove an individual metric, click the cancel icon (✖) next to the metric name.
   - To remove all metrics, click the cancel icon (✖) in the **Select a metric** list.
6. Optional: To return to configuring bias metrics for the model, on the metrics page, click **Configure** in the upper-right corner.

Verification

- The metrics page shows the metrics selections that you made.

4.4. USING BIAS METRICS

You can use the following bias metrics in Red Hat OpenShift AI:

Statistical Parity Difference

Statistical Parity Difference (SPD) is the difference in the probability of a favorable outcome prediction between unprivileged and privileged groups. The formal definition of SPD is the following:

$$SPD = p(\hat{y} = 1 \mid D_u) - p(\hat{y} = 1 \mid D_p)$$

- $\hat{y} = 1$ is the favorable outcome.
- D_u and D_p are the unprivileged and privileged group data.

You can interpret SPD values as follows:

- A value of **0** means that the model is behaving fairly for a selected attribute (for example, race, gender).
- A value in the range **-0.1** to **0.1** means that the model is reasonably fair for a selected attribute. Instead, you can attribute the difference in probability to other factors, such as the sample size.
- A value outside the range **-0.1** to **0.1** indicates that the model is unfair for a selected attribute.
- A negative value indicates that the model has bias against the unprivileged group.
- A positive value indicates that the model has bias against the privileged group.

Disparate Impact Ratio

Disparate Impact Ratio (DIR) is the ratio of the probability of a favorable outcome prediction for unprivileged groups to that of privileged groups. The formal definition of DIR is the following:

$$DIR = \frac{p(\hat{y} = 1 \mid D_u)}{p(\hat{y} = 1 \mid D_p)}$$

- $\hat{y} = 1$ is the favorable outcome.
- D_u and D_p are the unprivileged and privileged group data.

The threshold to identify bias depends on your own criteria and specific use case.

For example, if your threshold for identifying bias is represented by a DIR value below **0.8** or above **1.2**, you can interpret the DIR values as follows:

- A value of **1** means that the model is fair for a selected attribute.
- A value of between **0.8** and **1.2** means that the model is reasonably fair for a selected attribute.
- A value below **0.8** or above **1.2** indicates bias.

CHAPTER 5. MONITORING DATA DRIFT

Data drift refers to changes that occur in the distribution of incoming data that differ significantly from the data on which the model was originally trained. This distributional shift can cause model performance to become unreliable, because machine learning models rely heavily on the patterns in their training data.

Detecting data drift helps ensure that your models continue to perform as expected and that they remain accurate and reliable. Trusty AI measures the statistical alignment between a model's training data and its incoming inference data using specialized metrics.

Metrics for drift detection include:

- Mean-Shift
- FourierMMD
- Kolmogorov-Smirnov
- ApproxKSTest

5.1. CREATING A DRIFT METRIC

To monitor a deployed model for data drift, you must first create drift metrics.

For information about the specific data drift metrics, see Using drift metrics.

For the complete list of TrustyAI metrics, see TrustyAI service API.

5.1.1. Creating a drift metric by using the CLI

You can use the OpenShift CLI (**oc**) to create a data drift metric for a model.

Prerequisites

- You are familiar with the specific data set schema and understand the relevant inputs and outputs.
- Your OpenShift cluster administrator added you as a user to the OpenShift cluster and has installed the TrustyAI service for the project that contains the deployed models.
- You set up TrustyAI for your project, as described in Setting up TrustyAI for your project.

Procedure

1. Open a new terminal window.
2. Follow these steps to log in to your OpenShift cluster:
   a. In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**.
   b. After you have logged in, click **Display token**.
   c. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

3. Set the **TRUSTY_ROUTE** variable to the external route for the TrustyAI service pod.

```
TRUSTY_ROUTE=https://$(oc get route/trustyai-service --template={{.spec.host}})
```

4. Optionally, get the full list of TrustyAI service endpoints and payloads.

```
curl -H "Authorization: Bearer $TOKEN" --location $TRUSTY_ROUTE/q/openapi
```

5. Use **POST /metrics/drift/meanshift/request** to schedule a recurring drift monitoring metric with the following syntax and payload structure:
Syntax:

```
curl -k -H "Authorization: Bearer $TOKEN" -X POST --location
$TRUSTY_ROUTE/metrics/drift/meanshift/request \
 --header 'Content-Type: application/json' \
 --data <payload>
```

Payload structure:

modelId

The name of the model to monitor.

referenceTag

The data to use as the reference distribution.

For example:

```
curl -k -H "Authorization: Bearer $TOKEN" -X POST --location
$TRUSTY_ROUTE/metrics/drift/meanshift/request \
    --header 'Content-Type: application/json' \
    --data "{
                \"modelId\": \"gaussian-credit-model\",
                \"referenceTag\": \"TRAINING\"
            }"
```

5.2. DELETING A DRIFT METRIC BY USING THE CLI

You can use the OpenShift CLI (**oc**) to delete a drift metric for a model.

Prerequisites

- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
  - Installing the OpenShift CLI for OpenShift Container Platform
  - Installing the OpenShift CLI for Red Hat OpenShift Service on AWS
- You have a user token for authentication as described in Authenticating the TrustyAI service.
- There is an existing drift metric that you want to delete.

Procedure

1. Open a new terminal window.
2. Follow these steps to log in to your OpenShift cluster:
   a. In the OpenShift web console, click your user name and select **Copy login command**.
   b. After you have logged in, click **Display token**.
   c. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

   ```
   $ oc login --token=<token> --server=<openshift_cluster_url>
   ```

3. In the OpenShift CLI (**oc**), get the route to the TrustyAI service:

   ```
   TRUSTY_ROUTE=https://$(oc get route/trustyai-service --template={{.spec.host}})
   ```

4. Optional: To list all currently active requests for a metric, use **GET /metrics/{{metric}}/requests**. For example, to list all currently scheduled MeanShift metrics, type:

   ```
   curl -k -H "Authorization: Bearer $TOKEN" -X GET --location
   "$TRUSTY_ROUTE/metrics/drift/meanshift/requests"
   ```

   Alternatively, to list all currently scheduled metric requests, use **GET /metrics/all/requests**.

   ```
   curl -H "Authorization: Bearer $TOKEN" -X GET --location
   "$TRUSTY_ROUTE/metrics/all/requests"
   ```

5. To delete a metric, send an HTTP **DELETE** request to the **/metrics/$METRIC/request** endpoint to stop the periodic calculation, including the id of periodic task that you want to cancel in the payload. For example:

   ```
   curl -k -H "Authorization: Bearer $TOKEN" -X DELETE --location
   "$TRUSTY_ROUTE/metrics/drift/meanshift/request" \
       -H "Content-Type: application/json" \
       -d "{
             \"requestId\": \"$id\"
           }"
   ```

Verification

Use **GET /metrics/{{metric}}/requests** to list all currently active requests for the metric and verify the metric that you deleted is not shown. For example:

```
curl -H "Authorization: Bearer $TOKEN" -X GET --location
"$TRUSTY_ROUTE/metrics/drift/meanshift/requests"
```

5.3. VIEWING DRIFT METRICS FOR A MODEL

After you create data drift monitoring metrics, use the OpenShift web console to view and update the drift metrics that you configured.

Prerequisites

- You have been assigned the **monitoring-rules-view** role. For more information, see Granting users permission to configure monitoring for user-defined projects.
- You are familiar with how to monitor project metrics in the OpenShift web console. For more information, see Monitoring your project metrics.

Procedure

1. Log in to the OpenShift web console.
2. Switch to the **Developer** perspective.
3. In the left menu, click **Observe**.
4. As described in Monitoring your project metrics, use the web console to run queries for **trustyai_*** metrics.

5.4. USING DRIFT METRICS

You can use the following data drift metrics in Red Hat OpenShift AI:

MeanShift

The MeanShift metric calculates the per-column probability that the data values in a test data set are from the same distribution as those in a training data set (assuming that the values are normally distributed). This metric measures the difference in the means of specific features between the two datasets.
MeanShift is useful for identifying straightforward changes in data distributions, such as when the entire distribution has shifted to the left or right along the feature axis.

This metric returns the probability that the distribution seen in the "real world" data is derived from the same distribution as the reference data. The closer the value is to 0, the more likely there is to be significant drift.

FourierMMD

The FourierMMD metric provides the probability that the data values in a test data set have drifted from the training data set distribution, assuming that the computed Maximum Mean Discrepancy (MMD) values are normally distributed. This metric compares the empirical distributions of the data sets by using an MMD measure in the Fourier domain.
FourierMMD is useful for detecting subtle shifts in data distributions that might be overlooked by simpler statistical measures.

This metric returns the probability that the distribution seen in the "real world" data has drifted from the reference data. The closer the value is to 1, the more likely there is to be significant drift.

KSTest

The KSTest metric calculates two Kolmogorov-Smirnov tests for each column to determine whether the data sets are derived from the same distributions. This metric measures the maximum distance between the empirical cumulative distribution functions (CDFs) of the data sets, without assuming any specific underlying distribution.
KSTest is useful for detecting changes in distribution shape, location, and scale.

This metric returns the probability that the distribution seen in the "real world" data is derived from the same distribution as the reference data. The closer the value is to 0, the more likely there is to be significant drift.

ApproxKSTest

The ApproxKSTest metric performs an approximate Kolmogorov-Smirnov test, ensuring that the maximum error is **6*epsilon** compared to an exact KSTest.
ApproxKSTest is useful for detecting changes in distributions for large data sets where performing an exact KSTest might be computationally expensive.

This metric returns the probability that the distribution seen in the "real world" data is derived from the same distribution as the reference data. The closer the value is to 0, the more likely there is to be significant drift.

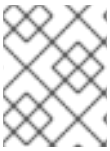
5.5. USING A DRIFT METRIC IN A CREDIT CARD SCENARIO

This example scenario deploys an XGBoost model into your cluster and reviews its output using a drift metric.

The XGBoost model was created for the purpose of this demonstration and predicts credit card approval based on the following features: age, credit score, years of education, and years in employment.

When the model is deployed and the data that you upload is formatted, use the mean shift metric to monitor for data drift. This metric is useful for ensuring that a model remains accurate and reliable in a production environment.

Mean shift compares a numeric test dataset against a numeric training dataset. It produces a p-value that measures the probability the test data has originated from the same numeric distribution as the training data. A p-value less than 0.05 indicates a statistically significant drift between the two datasets. A p-value equal to or greater than 0.05 indicates no statistically significant evidence of drift.



NOTE

Mean shift performs best when each feature in the data is normally distributed. Choose a different metric for working with different or unknown data distributions.

Prerequisites

- Your cluster administrator added you as a user to the cluster and configured TrustyAI for use in the project that contains the deployed models.
- You set up TrustyAI for your project, as described in Setting up TrustyAI for your project.

Procedure

1. Obtain a bearer token to authenticate your external endpoints by running the following command:

```
$ oc apply -f resources/service_account.yaml
export TOKEN=$(oc create token user-one)
```

2. In your model namespace, deploy the storage container, serving runtime, and the credit model:

```
$ oc project model-namespace || true
$ oc apply -f resources/model_storage_container.yaml
$ oc apply -f resources/odh-mlserver-1.x.yaml
$ oc apply -f resources/model_gaussian_credit.yaml
```

3. Set the route for your data upload:

```
TRUSTY_ROUTE=https://$(oc get route/trustyai-service --template={{.spec.host}})
```

4. Download the training data payload (file size 472 KB):

```
wget https://github.com/trustyai-explainability/odh-trustyai-demos/blob/72f748da9410f92a60bea73ce5e3f47c10ad1cea/3-DataDrift/kserve-demo/data/training_data.json -O training_data.json
```

5. Label your model training data. This data has four main fields. The **model_name** and **data_tag** fields require a label because they are directly referenced in the Metrics dashboard later in the scenario. In addition to the required fields, it is best to also label response and request fields. The four fields are:

   a. **model_name**: The name of the model that correlates to this data. The name should match that of the model provided in the model YAML, which is **gaussian-credit-model**.

   b. **data_tag**: A string tag to reference this particular set of data. Use the string **"TRAINING"**.

   c. **request**: This is a KServe inference request, as if you were sending this data directly to the model server's **/infer** endpoint.

   d. **response**: The KServe inference response that is returned from sending the above request to the model.

6. Upload the model training data to the TrustyAI endpoint:

```
curl -sk -H "Authorization: Bearer ${TOKEN}" $TRUSTY_ROUTE/data/upload  \
 --header 'Content-Type: application/json' \
 -d @training_data.json
```

   The following message appears confirming the data upload: **1000 datapoints successfully added to gaussian-credit-model data**.

7. Label your model input and output fields with the actual column names of the data in your KServe payloads. Send a JSON payload containing a simple set of **original-name** : **new-name pairs**, assigning new meaningful names to the input and output features of your model. A message that says "Feature and output name mapping successfully applied" appears if the request is successful:

```
curl -sk -H "Authorization: Bearer ${TOKEN}" -X POST --location
$TRUSTY_ROUTE/info/names \
  -H "Content-Type: application/json"   \
  -d "{
    \"modelId\": \"gaussian-credit-model\",
    \"inputMapping\":
      {
        \"credit_inputs-0\": \"Age\",
```

```
      \"credit_inputs-1\": \"Credit Score\",
      \"credit_inputs-2\": \"Years of Education\",
      \"credit_inputs-3\": \"Years of Employment\"
    },
  \"outputMapping\": {
    \"predict-0\": \"Acceptance Probability\"
  }
}"
```

TIP

Define name mappings in TrustyAI to assign memorable names to input or output names. These names can then be used in subsequent requests to the TrustyAI service.

8. Verify that TrustyAI has received the data by querying the **/info** endpoint:

```
curl -H "Authorization: Bearer ${TOKEN}" $TRUSTY_ROUTE/info | jq '.["gaussian-credit-model"].data.inputSchema'
```

9. The following output appears as a JSON file confirming that TrustyAI has successfully received the data:

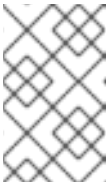
```
{
  "items": {
    "Years of Education": {
      "type": "DOUBLE",
      "name": "credit_inputs-2",
      "columnIndex": 2
    },
    "Years of Employment": {
      "type": "DOUBLE",
      "name": "credit_inputs-3",
      "columnIndex": 3
    },
    "Age": {
      "type": "DOUBLE",
      "name": "credit_inputs-0",
      "columnIndex": 0
    },
    "Credit Score": {
      "type": "DOUBLE",
      "name": "credit_inputs-1",
      "columnIndex": 1
    }
  },
  "nameMapping": {
    "credit_inputs-0": "Age",
    "credit_inputs-1": "Credit Score",
    "credit_inputs-2": "Years of Education",
    "credit_inputs-3": "Years of Employment"
  }
}
```

10. Create a recurring drift monitoring metric using **/metrics/drift/meanshift/request**. This will

measure the drift of all recorded inference data against the reference distribution. The body of the payload requires a **modelId** that sets which model to monitor and a referenceTag that determines which data to use as the reference distribution. The values of these fields should match the **modelId** and **referenceTag** inside your data upload payload:

```
curl -k -H "Authorization: Bearer ${TOKEN}" -X POST --location
$TRUSTY_ROUTE/metrics/drift/meanshift/request -H "Content-Type: application/json" \
 -d "{
      \"modelId\": \"gaussian-credit-model\",
      \"referenceTag\": \"TRAINING\"
    }"
```

11. Check the metrics in the OpenShift console under **Observe → Metrics**:
    a. Set the time window to 5 minutes and the refresh interval to 15 seconds.
    b. In the **Expression** field, enter **trustyai_meanshift**.



NOTE

It may take a few seconds before the cluster monitoring stacks picks up the new metric. You may need to refresh before the new metrics appear, if you're already in the section of the OpenShift console.

12. Observe in the Metric Chart onscreen that a metric is emitted for each of the four features and the single output, making for five measurements in total. All metric values should equal 1 (no drift), because we only have the training data, which can't drift from itself.
13. Collect some simulated real-world inferences to observe the drift monitoring. To do this, send small batches of data to the model, mimicking a real-world deployment:
    a. Get the route to the model:

```
MODEL=gaussian-credit-model
BASE_ROUTE=$(oc get inferenceservice gaussian-credit-model -o
jsonpath='{.status.url}')
MODEL_ROUTE="${BASE_ROUTE}/v2/models/${MODEL}/infer"
```

    b. Download the data batch and send data payloads to your model:

```
DATA_PATH=sample_trustyai_model_data
mkdir $DATA_PATH
for batch in {0..595..5}; do
 wget https://github.com/trustyai-explainability/odh-trustyai-
demos/blob/72f748da9410f92a60bea73ce5e3f47c10ad1cea/3-DataDrift/kserve-
demo/data/data_batches/$batch.json -O $DATA_PATH/$batch.json
 curl -sk "${MODEL_ROUTE}"\
   -H "Authorization: Bearer ${TOKEN}" \
   -H "Content-Type: application/json" \
   -d @$DATA_PATH/$batch.json
  sleep 1
done
```

14. Observe the updated drift metrics in the **Observe → Metrics** section of the OpenShift console. The mean shift metric values for the various features change:

    a. The values for Credit Score, Age, and Acceptance Probability have all dropped to 0, indicating there is a statistically very high likelihood that the values of these fields in the inference data come from a different distribution than that of the training data.

    b. The Years of Employment and Years of Education scores have dropped to 0.34 and 0.82 respectively, indicating that there is a little drift, but not enough to be particularly concerning.